COURSE OUTLINE · ACADEMIC YEAR 2026–2027

Computer Networks

Annual lecture and laboratory course for the Computer Science (Informatică) bachelor programme

1. Course at a glance

Format	Two semesters of 14 teaching weeks each. Every week: one 2-hour lecture and one 2-hour laboratory.
Contact hours	56 h lecture + 56 h laboratory over the year (28 + 28 per semester).
Level	Bachelor, second or third year, Computer Science profile.
Language	English.
Prerequisites	Programming in C and Python; Operating Systems (processes, files, system calls); Data Structures and Algorithms (graphs, shortest paths); Computer Architecture; elementary probability.
Semester 1	*The Internet protocol stack, top-down.* Applications, transport, network layer and routing. Laboratory centred on socket programming and packet analysis on Linux.
Semester 2	*Below IP and beyond the basics.* Link and physical layers, LANs, wireless and mobile networks, security, software-defined and data-centre networking, multimedia. Laboratory centred on switching, security and a team project.
Credits	To be fixed in the curriculum (*plan de învățământ*). The syllabi consulted assign between 3 and 6 ECTS to a one-semester networks course.

2. Aims and learning outcomes

The course explains how the Internet works, why it was designed the way it was, and how to write software that uses it correctly. By the end of the year a student should be able to:

1. Describe the layered architecture of the Internet and the service each layer offers to the one above it, using both the OSI and the TCP/IP reference models.
2. Explain the operation of the core protocols (HTTP, DNS, SMTP, TCP, UDP, QUIC, IPv4, IPv6, ICMP, ARP, DHCP, OSPF, BGP, Ethernet, 802.11, TLS) and read their messages in a packet capture.
3. Design and implement client–server and peer-to-peer applications with the BSD socket API, including concurrent servers and a reliable transport protocol built over UDP.
4. Plan IP addressing for a small organisation, and configure and troubleshoot switches, VLANs, routers, NAT, DHCP and DNS in an emulated network.
5. Reason quantitatively about delay, throughput, loss and congestion, and measure them with standard tools.
6. Identify common attacks on networks and apply cryptographic and operational defences (TLS, VPNs, firewalls).
7. Read a networking research paper and relate it to the principles taught in the course.

3. How this outline was aligned with other courses

The outline was built after reading the published syllabi (*fișa disciplinei*) or public course pages of the networks courses listed below. The table records what each source emphasises and where that emphasis appears in this course.

Institution and course	What the published material emphasises	Reflected here in
Babeș-Bolyai University, Cluj-Napoca — Rețele de calculatoare / Principles of Computer Networks	Socket programming (TCP, then UDP) taught before the theory; OSI and TCP/IP; HTTP, FTP, DNS, e-mail; TCP versus UDP and congestion control; distance-vector and link-state routing, RIP; IP addressing, subnetting and aggregation; NAT, packet filtering, VPN and VLAN. Labs: UDP and TCP client–server programs, then simulator work on LAN/WAN, routing, DHCP, DNS, NAT, wireless.	Sem. 1 weeks 3–13 (lectures and labs); Sem. 2 weeks 3, 6, 8–9.
University of Bucharest, Faculty of Mathematics and Computer Science — Computer Networks (public course repository)	A practical course built on Docker containers, Python and Scapy: command-line tools and Wireshark, protocol stacks, DNS and HTTP/1.1/2/3, UDP and TCP sockets, TCP congestion control, IPv4/IPv6/ICMP, subnetting and routing, Ethernet and ARP, packet interception, DHCP, DNS spoofing.	Lab environment (containers, Python and Scapy alongside C); Sem. 1 labs 1–2, 7, 10; Sem. 2 labs 2 and 10.
Alexandru Ioan Cuza University, Iași — Computer Networks	Network types; TCP/IP; architecture models; client/server paradigm; BSD socket interface; application protocols (terminal, SMTP, FTP, POP); RPC; peer-to-peer; wireless networks; security. Week 1 covers history and fundamental notions, and the first lab revises Operating Systems.	Lecture 1 and Lab 1; Sem. 1 weeks 3–6; Sem. 2 weeks 5, 7–9 and 13 (RPC).
Politehnica Bucharest (UNSTPB) — Protocoale de comunicație	Laboratories written in C/C++ on Linux, with a prepared virtual machine and a reading assigned before every lab. *(Only the laboratory front page was accessible; the lecture list was not.)*	Lab conventions: Linux VM, C as first language, pre-lab reading.
Technical University of Cluj-Napoca — Rețele de calculatoare	A bottom-up course: transmission techniques and encoding, physical media and structured cabling, medium access, data link layer (HDLC), LANs, interconnection devices, IP/ICMP/IPv6, routing, TCP/UDP, applications, security. Labs: Wireshark and Cisco Packet Tracer, IPv4, routing and DHCP, IPv6, sockets, Ethernet/ARP/NDP, VLANs and trunking, spanning tree and link aggregation, security threats, final lab test.	Sem. 2 weeks 1–6 (lectures and labs); lab tests; Packet Tracer in Sem. 2.
Politehnica University of Timișoara — Rețele de calculatoare	Definitions, services and classification of networks; topologies; client–server and peer-to-peer; theoretical models and practical architectures; Ethernet and Fast Ethernet; design, configuration and administration. *(Only fragments of the syllabus were retrieved, from the Hunedoara engineering faculty.)*	Lecture 1 (classification, topologies); Sem. 2 weeks 3 and 6.
Stanford University — CS 144 Introduction to Computer Networking	Learning by building: across eight lab checkpoints students implement a working TCP/IP stack in C++, from a byte-stream reassembler up to a router. Lectures start from datagrams, encapsulation and multiplexing, then “reliability from unreliability”. Optional texts: Kurose & Ross, Peterson & Davie, Saltzer & Kaashoek. Labs carry about 38 % of the grade, exams 40 %.	Programming assignments A1–A4 (section 7); the weight given to practical work; the textbook list.

Institution and course	What the published material emphasises	Reflected here in
MIT — 6.829 Computer Networks (graduate)	Architectural principles, transport, Internet routing, router design, congestion control, wireless, security, naming, overlay and peer-to-peer networks; in recent editions also data-centre networks, software-defined networking, programmable data planes, content distribution and video streaming. Readings from original research papers; semester-long project.	Sem. 2 weeks 10–14; the optional paper readings; the Semester 2 team project.

How the six Romanian universities were chosen, and what was not verified

There is no single agreed "top six" for computer science. The Times Higher Education 2026 subject ranking for Computer Science lists Babeș-Bolyai, the Technical University of Cluj-Napoca, the University of Bucharest and Ștefan cel Mare Suceava in the 801–1000 band, and Alexandru Ioan Cuza Iași and Politehnica Timișoara among those in 1001+. Five of the six above come from that list. Politehnica Bucharest does not appear in it for Computer Science; it was included by judgement, as the country's largest computing faculty. Replacing it with Suceava would follow the ranking strictly.

For Politehnica Bucharest and Politehnica Timișoara only part of the course material could be read, as marked in the table. Those two rows should be checked against the full syllabi before this document is cited formally.

4. Design choices

- **Top-down first, bottom-up second.** Semester 1 starts from applications students already use and descends to routing, following Kurose & Ross and the Stanford sequence. Semester 2 covers the link and physical layers in the depth that the engineering-school syllabi give them, then moves to security and modern topics.
- **Sockets early.** Socket programming begins in week 3, as at Babeș-Bolyai and Iași, so that every later protocol can be observed from a program the student wrote.
- **Two kinds of laboratory work.** Programming on Linux (C, then Python and Scapy) and configuration in an emulator (Mininet, Cisco Packet Tracer). Romanian syllabi split between these two traditions; the course uses both.
- **Build it to understand it.** Four programming assignments ask students to implement pieces of the stack themselves, in the spirit of Stanford's lab checkpoints, at a size that fits a 14-week semester.
- **A window onto research.** From Semester 2 week 10 each lecture has one optional classic or recent paper, as MIT's graduate course does throughout.

5. Semester 1 — The Internet protocol stack, top-down

Readings refer to chapters of Kurose & Ross (KR), Peterson & Davie (PD) and Tanenbaum, Feamster & Wetherall (TFW); see section 9.

Wk	Lecture (2 h)	Laboratory (2 h)
1	**Introduction.** What a network and a protocol are; network edge, access networks and core; packet versus circuit switching; the Internet as a network of networks; a first look at delay and at layering; short history. *KR 1 · PD 1 · TFW 1*	**Lab environment.** Linux refresher, the course VM and containers. `ip`, `ping`, `traceroute`, `ss`, `dig`, `curl`. Who is my default gateway, and how far away is a server?
2	**Architecture and performance.** Layering and encapsulation; OSI and TCP/IP reference models; standards bodies and RFCs; delay, loss and throughput in depth; traffic intensity; end-to-end argument. *KR 1 · PD 1 · TFW 1*	**Packet analysis.** Wireshark and `tcpdump`; capture and display filters; following one web request down the stack; measuring RTT.
3	**Application layer I.** Application architectures; processes, ports and sockets; services applications need from transport; the Web and HTTP/1.1: messages, methods, status codes, persistent connections. *KR 2 · PD 9*	**Sockets I: TCP.** `socket`, `connect`, `bind`, `listen`, `accept`, `send`, `recv` in C; an echo client and server; partial reads and message framing.
4	**Application layer II.** Cookies and state; web caching and conditional GET; HTTP/2 multiplexing; HTTP/3 over QUIC; content distribution networks. *KR 2 · PD 9*	**Sockets II: UDP.** `sendto`/`recvfrom`; byte order and serialisation; timeouts; the same service over TCP and UDP compared. Python sockets.
5	**Naming: the Domain Name System.** Hierarchy and delegation; resource records; iterative and recursive resolution; caching and TTL; DNS as a load-balancing and CDN tool; first look at DNS attacks. *KR 2 · TFW 7*	**Concurrent servers.** One process per client, threads, and I/O multiplexing with `select`/`poll`/`epoll`; a multi-client chat server.
6	**Other applications.** E-mail: SMTP, IMAP, message format; remote login and file transfer (SSH, FTP); peer-to-peer file sharing, BitTorrent, distributed hash tables. *KR 2 · TFW 7*	**HTTP by hand.** Talking HTTP over a raw TCP socket; a minimal HTTP/1.1 server serving static files; testing with `curl` and a browser. *Assignment A1 released.*
7	**Transport I.** Transport services; multiplexing and demultiplexing; UDP and the Internet checksum; principles of reliable data transfer: ACKs, timers, sequence numbers, stop-and-wait. *KR 3 · PD 5*	**DNS in practice.** `dig` and the delegation chain; building and parsing a DNS query over UDP by hand; observing caching.
8	**Transport II.** Pipelining; Go-Back-N and Selective Repeat; TCP segment structure, sequence and acknowledgement numbers, RTT estimation and retransmission timeout, connection set-up and tear-down. *KR 3 · PD 5*	**Lab test 1** (sockets, 50 min). Then: designing your own application-layer protocol — message grammar, state machine, error handling. *A1 due.*
9	**Transport III.** TCP flow control; causes and costs of congestion; AIMD, slow start, fast retransmit and recovery; TCP Reno, CUBIC and BBR; fairness; QUIC as a transport. *KR 3 · PD 6*	**Reliable transfer over UDP.** Stop-and-wait, then a sliding window, tested on a lossy link emulated with `tc netem`. *Assignment A2 released.*
10	**Network layer I: data plane.** Forwarding versus routing; inside a router; IPv4 datagram; fragmentation; addressing, subnets, CIDR and longest-prefix match. *KR 4 · PD 3*	**TCP under the microscope.** Handshake, retransmissions and window growth in Wireshark; `iperf3` throughput experiments under loss and delay in Mininet.
11	**Network layer II.** DHCP; NAT and its consequences; ICMP, `ping` and `traceroute` explained; IPv6 addressing, header and neighbour discovery; IPv4–IPv6 transition. *KR 4 · PD 3–4*	**IP addressing.** Subnetting and aggregation exercises; building a three-subnet topology with Linux network namespaces; static addresses and routes. *A2 due.*

Wk	Lecture (2 h)	Laboratory (2 h)
12	**Routing algorithms.** The network as a graph; link-state routing and Dijkstra; distance-vector routing and Bellman–Ford; count-to-infinity and poisoned reverse; hierarchical routing. *KR 5 · PD 3*	**A router's data plane.** Implementing forwarding in software: longest-prefix match, TTL and checksum update, ARP, ICMP errors. *Assignment A3 released.*
13	**Routing in the Internet.** Autonomous systems; OSPF; BGP: path attributes, route selection, policy, customer–provider and peering relationships; routing incidents. *KR 5 · PD 4*	**Dynamic routing, NAT, DHCP.** OSPF with FRRouting in an emulated topology; watching reconvergence after a link failure; configuring NAT and a DHCP server.
14	**Synthesis.** A day in the life of a web request, from DHCP to the rendered page; preview of the link layer (ARP, Ethernet, switches); review and exam preparation. *KR 6 (overview)*	**Lab test 2** (practical, 50 min) and oral defence of assignment A3. *A3 due.*

6. Semester 2 — Below IP, security and modern networks

Wk	Lecture (2 h)	Laboratory (2 h)
1	**Link layer I.** Link-layer services; framing and byte/bit stuffing; error detection and correction: parity, checksums, CRC, Hamming codes; HDLC and PPP as examples. *KR 6 · TFW 3*	**Error control codes.** Implementing CRC-32 and a Hamming(7,4) encoder/decoder; injecting bit errors and measuring detection rates.
2	**Medium access control.** Channel partitioning (TDMA, FDMA, CDMA); random access: ALOHA, slotted ALOHA, CSMA, CSMA/CD; taking-turns protocols; efficiency analysis. *KR 6 · TFW 4*	**Ethernet and ARP with Scapy.** Crafting and sniffing frames; the ARP exchange; MAC addresses and broadcast domains.
3	**Switched LANs.** Ethernet frame and standards; MAC addressing and ARP; self-learning switches; spanning tree protocol; VLANs and 802.1Q trunking; link aggregation. *KR 6 · PD 2–3*	**Switching I.** A learning switch in Mininet; VLANs, trunks and inter-VLAN routing in Packet Tracer.
4	**Physical layer.** Signals and bandwidth; Nyquist and Shannon limits; guided media (twisted pair, coaxial, fibre) and structured cabling; line coding and modulation; multiplexing. *TFW 2 · Stallings*	**Switching II.** Spanning tree: root election, blocked ports and reconvergence; EtherChannel/link aggregation; loop experiments.
5	**Wireless LANs.** Wireless link characteristics; hidden terminal problem; IEEE 802.11 architecture, CSMA/CA, RTS/CTS, frame format, rate adaptation; Wi-Fi 6/7 in brief; Bluetooth. *KR 7 · PD 2*	**Wi-Fi.** Analysing 802.11 captures (beacons, association, data, ACKs); access-point configuration; WPA2/WPA3 settings. *Project teams and topics fixed.*
6	**Cellular and mobile networks.** 4G/5G architecture; radio access and core; mobility management, handover and roaming; Mobile IP principles; impact on transport and applications. *KR 7*	**Integrated network design.** A small campus network in Packet Tracer: addressing plan, VLANs, DHCP, DNS, NAT, OSPF; systematic troubleshooting.

Wk	Lecture (2 h)	Laboratory (2 h)
7	**Security I: foundations.** Threat model; symmetric and public-key cryptography; hash functions and message integrity; digital signatures; certificates and public-key infrastructure. *KR 8 · PD 8*	**Lab test 3** (configuration, 50 min). **Project milestone 1:** design review.
8	**Security II: secure protocols.** TLS 1.3 handshake and record protocol; HTTPS; IPsec and VPNs; WireGuard; securing e-mail; DNSSEC and encrypted DNS; Wi-Fi security. *KR 8 · PD 8*	**Cryptography in practice.** OpenSSL: keys, certificates, a private CA; inspecting a TLS 1.3 handshake in Wireshark; TLS-protected sockets.
9	**Security III: attacks and defences.** Sniffing, spoofing, ARP and DNS poisoning, DDoS and amplification; firewalls and packet filtering; intrusion detection; BGP hijacks and RPKI. *KR 8 · TFW 8*	**Attack and defence in an isolated lab.** ARP and DNS spoofing with Scapy inside containers; writing `nftables` firewall rules; an SSH tunnel and a WireGuard VPN.
10	**Software-defined networking and management.** Separating control and data planes; OpenFlow and match–action forwarding; programmable data planes (P4); SNMP, NETCONF/YANG, telemetry. *KR 4–5 · paper: McKeown et al., "OpenFlow", 2008*	**SDN.** Mininet with an OpenFlow controller: a learning switch, then a simple firewall and a load balancer as controller applications.
11	**Data-centre networking.** Fat-tree and leaf–spine topologies; ECMP; virtual networks and overlays (VXLAN); data-centre transport (DCTCP); load balancing. *KR 6 · paper: Al-Fares et al., "A Scalable, Commodity Data Center Network Architecture", 2008*	**Measurement and queue management.** `iperf3`, `tc` queueing disciplines and traffic shaping; observing bufferbloat and the effect of active queue management. **Project milestone 2.**
12	**Multimedia networking and quality of service.** Streaming stored video and DASH; voice and video conferencing, RTP and WebRTC; jitter and loss recovery; scheduling, policing, DiffServ. *PD 6–7 · paper: Jacobson, "Congestion Avoidance and Control", 1988*	**Distributed programming.** Remote procedure call with gRPC; comparing a hand-written socket protocol with an RPC interface; latency measurements.
13	**Overlays and large-scale services.** RPC and its semantics; overlay networks; structured peer-to-peer systems (Chord); CDNs and anycast in depth; network neutrality and Internet governance. *PD 9 · paper: Stoica et al., "Chord", 2001*	**Network automation.** Configuring devices from Python; configuration as code; project integration and testing.
14	**Frontiers and review.** QUIC and HTTP/3 internals; the Internet of Things; low-Earth-orbit satellite networks; open problems; course review and exam preparation. *paper: Clark, "The Design Philosophy of the DARPA Internet Protocols", 1988*	**Project demonstrations** and oral defence.

7. Assignments and project

	Task	Weeks
A1	A concurrent HTTP/1.1 server in C supporting persistent connections, static files and correct status codes.	S1: 6–8
A2	A reliable file-transfer protocol over UDP with a sliding window, adaptive retransmission timeout and a short design report.	S1: 9–11

	Task	Weeks
A3	The data plane of an IPv4 router: longest-prefix match, ARP, ICMP, running in an emulated topology.	S1: 12–14
A4	A secure messaging application: your own application protocol carried over TLS, with certificate verification.	S2: 8–10
Project	Teams of two or three. One topic from a list (for example an SDN load balancer, a DNS resolver with caching, a simplified BitTorrent client, a congestion-control comparison study, a small campus network designed and documented end to end) or an approved proposal. Three checkpoints: design review, working prototype, final demonstration with a written report.	S2: 5–14

8. Assessment (proposal)

The same scheme applies in each semester, and each semester is graded separately on the 1–10 scale.

Component	Weight	Details
Written examination	50 %	End-of-semester exam: short conceptual questions and quantitative problems.
Laboratory	30 %	Weekly lab work (10 %) and the practical lab tests (20 %).
Assignments / project	20 %	Semester 1: A1–A3. Semester 2: A4 and the team project.

Minimum standard: at least 5 in the written examination and at least 5 in the laboratory component, the convention used in the Romanian syllabi consulted. Attendance at laboratories is compulsory. The weights are a starting point; the Romanian syllabi range from 50/50 to 60/40 between examination and practical work, and Stanford gives about 40 % to examinations.

9. Bibliography

Core textbooks

1. J. F. Kurose, K. W. Ross, *Computer Networking: A Top-Down Approach*, 8th edition, Pearson, 2021.
2. L. L. Peterson, B. S. Davie, *Computer Networks: A Systems Approach*, 6th edition, Morgan Kaufmann, 2021. Free online: book.systemsapproach.org.
3. A. S. Tanenbaum, N. Feamster, D. Wetherall, *Computer Networks*, 6th edition, Pearson, 2021. (Earlier editions are available in Romanian as *Rețele de calculatoare*.)

Supplementary

1. W. R. Stevens, B. Fenner, A. M. Rudoff, *UNIX Network Programming, Volume 1: The Sockets Networking API*, 3rd edition, Addison-Wesley, 2004.
2. W. Stallings, *Data and Computer Communications*, 10th edition, Pearson, 2014.
3. J. H. Saltzer, M. F. Kaashoek, *Principles of Computer System Design: An Introduction*, Morgan Kaufmann, 2009 (network chapters free online from MIT OpenCourseWare).

4. V. Dădârlat, E. Cebuc, *Rețele locale de calculatoare – de la cablare la interconectare*, Editura Albastră, Cluj-Napoca, 2006.
5. B. Hall, *Beej's Guide to Network Programming* (free online).
6. The RFC series, rfc-editor.org, for every protocol studied.

10. Sources consulted

Accessed on 5 October 2026.

- Babeș-Bolyai University, *Fișa disciplinei – Principiile rețelelor de calculatoare* (MLR5002), cs.ubbcluj.ro/files/curricula/2023/syllabus/
- Technical University of Cluj-Napoca, *Fișa disciplinei – Rețele de calculatoare* (code 39), 2026–2027, ac.utcluj.ro
- Alexandru Ioan Cuza University of Iași, Faculty of Computer Science, *Computer Networks 2026–2027*, edu.info.uaic.ro/computer-networks/
- University of Bucharest, FMI, *Computer Networks* course repository, github.com/senisioi/computer-networks
- Politehnica Bucharest, *Protocoale de comunicație*, ocw.cs.pub.ro/courses/pc and pcom.pages.upb.ro/labs/
- Politehnica University of Timișoara, *Fișa disciplinei – Rețele de calculatoare*, fih.upt.ro (search-result extracts)
- Stanford University, *CS 144: Introduction to Computer Networking*, Fall 2026, cs144.github.io
- MIT, *6.829 Computer Networks*, subject listing and Fall 2020 course page, web.mit.edu/6.829/www/2020
- Edupedu.ro, *Ranking Times Higher Education după domenii de studiu 2026*, 21 January 2026